



Window to Viewport Transformation

— Lecture 10

Window to Viewport Transformation



- **World coordinate** - It is the Cartesian coordinate w.r.t. which we define the diagram, like X_{wmin} , X_{wmax} , Y_{wmin} , Y_{wmax}
- **Device Coordinate**—It is the screen coordinate where the objects are to be displayed, like X_{vmin} , X_{vmax} , Y_{vmin} , Y_{vmax}
- **Window**—It is the area on the world coordinate selected for display.
- **Viewport**—It is the area on the device coordinate where graphics is to be displayed

আগে বুঝো — সমস্যাটা কী?

ধরো তুমি একটা বিশাল মানচিত্র (map) বানাতে কম্পিউটারে — পুরো বাংলাদেশের। কিন্তু তোমার স্ক্রিন ছোট। তাহলে কীভাবে দেখাবে?

অথবা, তুমি শুধু ঢাকার অংশটুকু দেখাতে চাও — পুরো মানচিত্র না।

এই সমস্যার সমাধানই হলো **Window to Viewport Transformation**।

Window to Viewport



Transformation

- It is the process of transforming 2D world-coordinate objects to device coordinates. Objects inside the world or clipping window are mapped to the viewport, which is the area on the screen where world coordinates are mapped to be displayed.
- In other words, the clipping window is used to select the part of the scene that is to be displayed. The viewport is used to display selected portion of window on the output device.



সহজ বাস্তব উদাহরণ (ক্যামেরা ও ছবি)

মনে করুন, আপনি আপনার স্মার্টফোনের ক্যামেরা দিয়ে একটি বিশাল ফুটবল মাঠের ছবি তুলছেন।

1. **Window** (ক্যামেরার ফ্রেম): বিশাল মাঠের পুরোটা কিন্তু ক্যামেরায় একবারে আসবে না। আপনি ফোনের স্ক্রিনে মাঠের যতটুকু অংশ ফ্রেমবন্দী বা সিলেক্ট করছেন, সেটাই হলো **Window** (বা **Clipping Window**)।
2. **Viewport** (প্রিন্ট করা ছবি বা ডিসপ্লে স্ক্রিন): তোলা ছবিটা যখন আপনি ফোনের গ্যালারিতে দেখছেন বা কাগজে ১০ সেন্টিমিটার সাইজের প্রিন্ট বের করছেন, সেই দেখানোর নির্দিষ্ট জায়গাটি হলো **Viewport**।

Window নির্ধারণ করে আপনি কী দেখতে চান (What to see), আর **Viewport** নির্ধারণ করে সেটি স্ক্রিনের কোথায় দেখা যাবে (Where to display)।



স্লাইডের লেখাগুলোর বাংলা ব্যাখ্যা

১ম পয়েন্ট:

"It is the process of transforming 2D world-coordinate objects to device coordinates. Objects inside the world or clipping window are mapped to the viewport, which is the area on the screen where world coordinates are mapped to be displayed."

- ব্যাখ্যা:
 - কম্পিউটার গ্রাফিক্সের আসল ওয়ার্ল্ড বা ভার্চুয়াল জগতে অবজেক্টগুলো থাকে **World Coordinates**-এ (যেমন: মিটারে বা ফিটে)।
 - কিন্তু আমাদের মনিটর বা ডিভাইসের নিজস্ব পিক্সেল সাইজ থাকে, যাকে বলে **Device Coordinates** (যেমন: 1920×1080 পিক্সেল)।
 - World Coordinate-এর অবজেক্টকে কনভার্ট বা স্কেল করে ডিভাইসের পিক্সেল অনুযায়ী স্ক্রিনের **Viewport** নামক জায়গায় বসিয়ে দেখানোর এই পুরো ম্যাপিং (Mapping) প্রসেসটিকেই **Window to Viewport Transformation** বলে।

২য় পয়েন্ট:

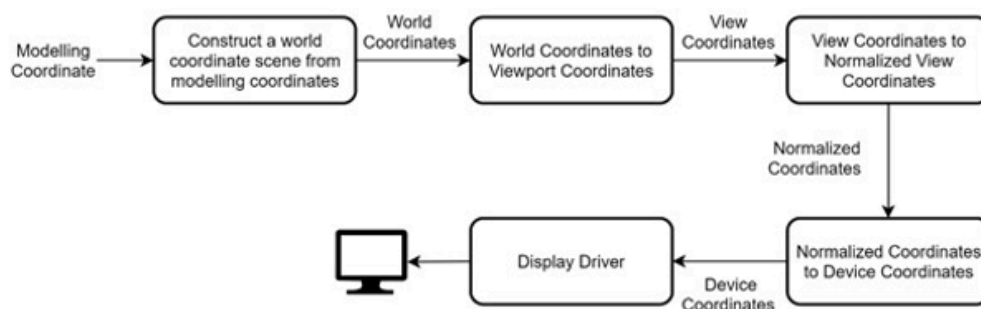
"In other words, the clipping window is used to select the part of the scene that is to be displayed. The viewport is used to display selected portion of window on the output device."

- ব্যাখ্যা:
 - **Clipping Window:** পুরো দৃশ্য বা সিন (Scene)-এর যতটুকু অংশ স্ক্রিনে প্রদর্শন করার জন্য পছন্দ/সিলেক্ট করা হয়, তা নির্ধারণ করে ক্লিপিং উইন্ডো।
 - **Viewport:** আউটপুট ডিভাইসের (যেমন: মনিটর, টিভি বা মোবাইল স্ক্রিন) যে নির্দিষ্ট অংশটিতে সেই সিলেক্ট করা ছবি বা দৃশ্যটি বাস্তবে ফুটিয়ে তোলা হয়, সেটিই হলো ভিউপোর্ট।

🔑 সংক্ষেপে মনে রাখার টিপস:

- **Window:** ওয়ার্ল্ড কোঅর্ডিনেট সিস্টেমে সিলেক্ট করা অঞ্চল (World Coordinate System)।
- **Viewport:** ডিসপ্লে স্ক্রিনের ওপর দেখানোর নির্দিষ্ট অঞ্চল (Device Coordinate System)।
- **Transformation:** উইন্ডোর সাইজ ছোট-বড় করে ভিউপোর্টে ফিট করা (Scaling & Translating)।

2D Viewing Transformation Pipeline



Coordinate System (কো-অর্ডিনেট সিস্টেম) হলো এমন একটি **reference system**, যার সাহায্যে কোনো point বা object-এর অবস্থান (**position**) নির্দিষ্ট করা হয়।

সহজভাবে বললে, কোনো point কোথায় আছে সেটা বোঝানোর জন্য coordinate system ব্যবহার করি।

সবচেয়ে **common: Cartesian Coordinate System**

এখানে দুইটি axis থাকে:

- **X-axis** → ডান-বাম অবস্থান
- **Y-axis** → উপর-নিচ অবস্থান

অবশ্যই। এটা মূলত **Computer Graphics**-এর **Coordinate Transformation Pipeline**—মানে একটা object/model তৈরি করার পর সেটা কীভাবে ধাপে ধাপে **screen/monitor**-এ দেখা যায়, সেটা দেখানো হয়েছে।

সহজভাবে পুরো flow-টা:

Model → World → Viewport → Normalized → Device → Display

1. Modelling Coordinates

প্রথমে আমরা object তৈরি করি।

যেমন ধরো তুমি একটা **car** বানাতে।

এখানে car-এর নিজের coordinate system থাকে, যেটাকে **Modelling Coordinates** বলে।

👉 সহজ কথা: **Object** কোথায়/কেমন তৈরি করেছে।

2. World Coordinates

এরপর সেই object-কে পুরো virtual world-এর মধ্যে বসানো হয়।

ধরো তুমি একটা game বানাচ্ছে:

- Car → (10, 20)
- Tree → (50, 30)
- House → (80, 40)

সব object-এর অবস্থান এখন একটি common coordinate system-এ।

এটাই **World Coordinates**।

Modelling Coordinates → World Coordinates

👉 Object-এর নিজের জায়গা থেকে পুরো পৃথিবীর/**scene**-এর জায়গায় নিয়ে আসা।

3. View Coordinates

এখন camera কোথায় আছে সেটা বিবেচনা করা হয়।

ধরো পুরো world অনেক বড়, কিন্তু camera শুধু একটা নির্দিষ্ট অংশ দেখছে।

তাই world-এর coordinate-গুলোকে **camera/view**-এর **perspective** অনুযায়ী পরিবর্তন করা হয়।

এটাকে বলা হয় **View Coordinates**।

👉 সহজভাবে:

World = পুরো **scene**

View = camera যেটা দেখছে

4. Normalized View Coordinates

এখন View Coordinates-কে একটা standard/normalized range-এর মধ্যে নিয়ে আসা হয়।

সাধারণত coordinate-গুলোকে এমন একটা standard system-এ transform করা হয় যাতে বিভিন্ন display/device-এর জন্য ব্যবহার করা সহজ হয়।

এটাকে বলা হয়:

Normalized View Coordinates

👉 সহজ কথা:

Camera-এর দেখা অংশকে **standard coordinate system**-এ **convert** করা।

5. Device Coordinates

এরপর normalized coordinates-কে নির্দিষ্ট display device-এর coordinate-এ convert করা হয়।

যেমন monitor-এর resolution:

1920 × 1080

এখন object-এর position বাস্তব pixel position-এ convert হবে।

যেমন:

Normalized position → Pixel position

এটাই **Device Coordinates**।

6. Display Driver

সবশেষে **Display Driver** এই device coordinates ব্যবহার করে monitor-এ actual image তৈরি করে।

অর্থাৎ:

Device Coordinates → Display Driver → Monitor

এখন তুমি screen-এ object-টা দেখতে পাচ্ছে।

পুরো ব্যাপারটা এক লাইনে মনে রাখো 👉

Modelling → World → View → Normalized → Device → Display

আর প্রত্যেকটার কাজ:

Stage	সহজ অর্থ
Modelling Coordinates	Object নিজে কোথায়
World Coordinates	পুরো world-এর মধ্যে object কোথায়
View Coordinates	Camera-এর দৃষ্টিতে object কোথায়
Normalized View Coordinates	Standard coordinate system
Device Coordinates	Screen/pixel-এর coordinate
Display Driver	Monitor-এ দেখায়

১. Modeling Coordinates → World Coordinates

- **Modeling Coordinates (MC):** এটি হলো কোনো অবজেক্টের নিজস্ব একক বা নিজস্ব ডিজাইন (যেমন: একটি গাড়ি বা বাড়ির মূল ডিজাইন, যা তার নিজস্ব কেন্দ্রকে (0, 0) ধরে তৈরি করা হয়)।
- কাজ: ডিজাইনারের তৈরি করা বিভিন্ন অবজেক্টকে এনে একটি মূল দৃশ্য (Scene) তাদের নির্দিষ্ট জায়গায় বসানো হয়।
- আউটপুট → **World Coordinates (WC):** পুরো ওয়ার্ল্ড বা দৃশ্যটির একটি সাধারণ কোঅর্ডিনেট সিস্টেম তৈরি হয় (যেমন: বাড়িটি কোথায় থাকবে, গাড়িটি কত দূরে থাকবে ইত্যাদি)।

২. World Coordinates → View Coordinates

- কাজ: ক্যামেরার চোখ বা ভিউয়ারের (Viewer) দৃষ্টিভঙ্গি অনুসারে সিনটিকে সাজানো হয়।
- আপনি ক্যামেরাকে কোন কোণে ধরেছেন বা উইন্ডোর (Window) মাধ্যমে কতটুকু অংশ দেখতে চাচ্ছেন—তার ওপর নির্ভর করে ওয়ার্ল্ড কোঅর্ডিনেটকে ভিউ কোঅর্ডিনেটে রূপান্তর করা হয়।
- আউটপুট → **View Coordinates (VC):** ভিউয়ার বা ক্যামেরার সাপেক্ষে অবজেক্টের অবস্থান।

৩. View Coordinates → Normalized View Coordinates

- কাজ: বিভিন্ন ডিসপ্লে ডিভাইসের সাইজ ও রেজোলিউশন ভিন্ন হয়। তাই সব হিসেবকে একটি স্ট্যান্ডার্ড বা সাশ্রয়ী সীমার মধ্যে আনার জন্য মানগুলোকে 0 থেকে 1 (অথবা -1 থেকে 1) রেঞ্জের মধ্যে স্কেল করা হয়।
- আউটপুট → **Normalized Coordinates (NC)**: ডিভাইস-নিরপেক্ষ (Device-independent) একটি মান, যা যেকোনো স্ক্রিনে সহজে ফিট করা যায়।

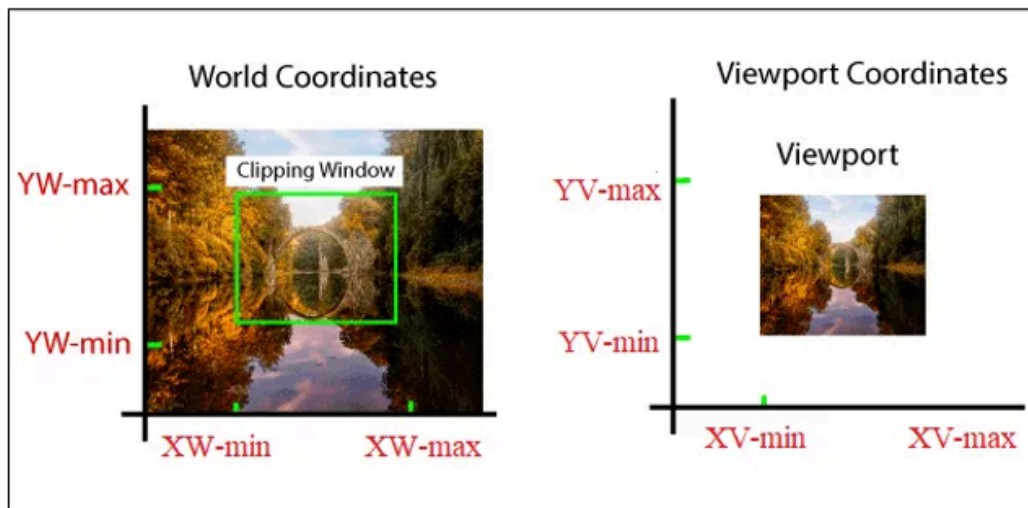
৪. Normalized Coordinates → Device Coordinates

- কাজ: নর্মালাইজড মানগুলোকে এবার বাস্তবে যে স্ক্রিনে ছবিটি দেখানো হবে, সেই স্ক্রিনের পিক্সেল রেঞ্জ (যেমন: 1920×1080 বা 1280×720) কনভার্ট করা হয়।
- আউটপুট → **Device Coordinates (DC)**: মনিটরের পিক্সেল পজিশন।

৫. Device Coordinates → Display Driver (Screen)

- কাজ: ডিভাইস কোঅর্ডিনেটের মানগুলো Display Driver-এ পাঠানো হয়, যা মনিটর বা স্ক্রিনের নির্দিষ্ট পিক্সেলে আলো জ্বেলে চূড়ান্ত ছবিটি আমাদের চোখে প্রদর্শন করে।

Window to Viewport Transformation



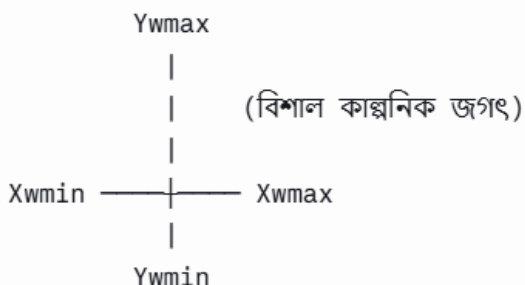
1 চারটো মূল ধারণা

World Coordinate (ওয়ার্ল্ড কো-অর্ডিনেট):

সহজ সংজ্ঞা: তুমি যে কাল্পনিক জগতে object বানাও, সেই জগতের coordinate system। এটা গণিতের Cartesian coordinate-এর মতো।

এখানে সীমানা নির্ধারণ হয়:

- **Xwmin, Xwmax** → X-অক্ষে বাম ও ডান সীমা
- **Ywmin, Ywmax** → Y-অক্ষে নিচ ও উপর সীমা



🌐 উদাহরণ: Google Maps-এ পুরো পৃথিবীর ডেটা আছে — এটা World Coordinate। কিন্তু তুমি সব দেখছো না।

🪟 Window (উইন্ডো):

সহজ সংজ্ঞা: World Coordinate-এর মধ্যে যে নির্দিষ্ট অংশটুকু দেখাতে চাও — সেটাই Window।

এটা একটা আয়তক্ষেত্র (rectangle) যা World Coordinate-এ থাকে।

বিশাল World Coordinate:



← শুধু এই অংশটুকু দেখাবো

🔍 উদাহরণ: Google Maps-এ zoom করে শুধু ঢাকা দেখছো — ঢাকার অংশটাই "Window"।

📱 Device Coordinate (ডিভাইস কো-অর্ডিনেট):

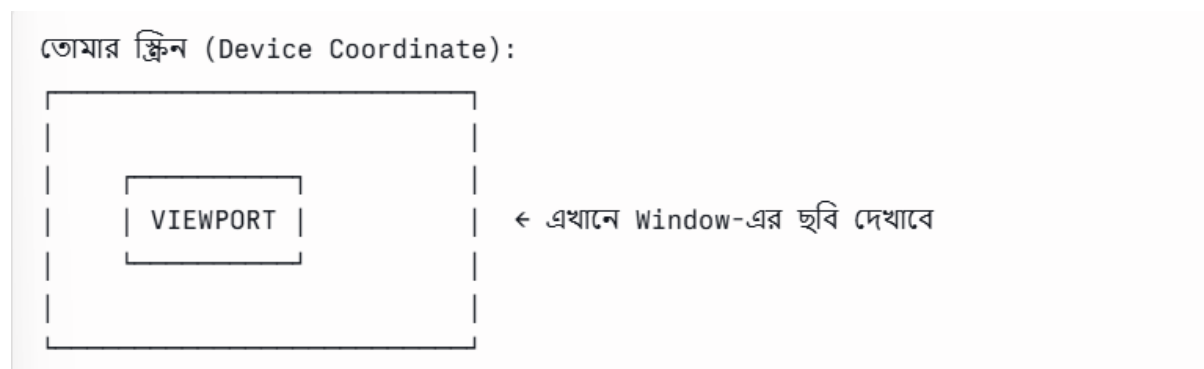
সহজ সংজ্ঞা: তোমার স্ক্রিনের coordinate system। এটা pixel-এ হিসাব হয়।

- **Xvmin, Xvmax** → স্ক্রিনে X-এর সীমা
- **Yvmin, Yvmax** → স্ক্রিনে Y-এর সীমা

উদাহরণ: তোমার মনিটর যদি 1920×1080 pixel হয়, তাহলে $X_{vmax}=1920$, $Y_{vmax}=1080$ ।

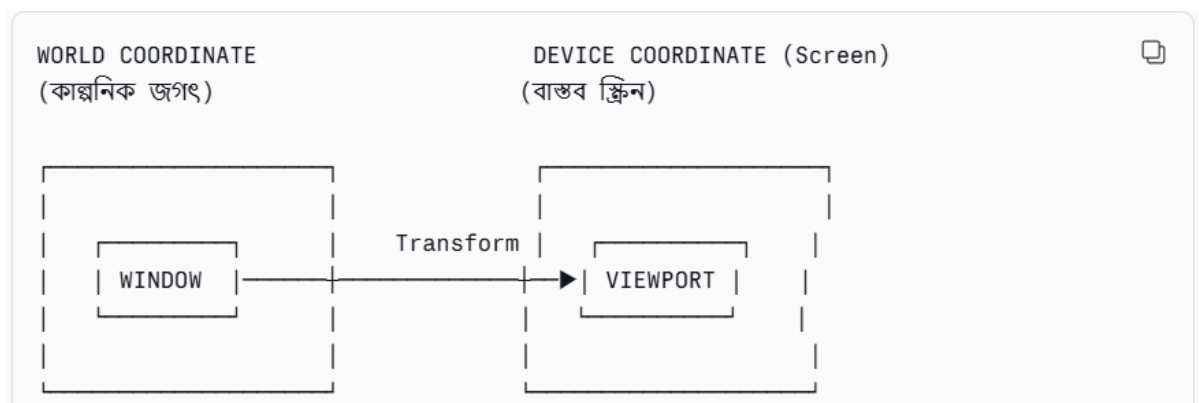
Viewport (ভিউপোর্ট):

সহজ সংজ্ঞা: Device Coordinate-এ যে নির্দিষ্ট এলাকায় Window-এর ছবি দেখাবে — সেটাই Viewport।



 উদাহরণ: ফোনে Google Maps অ্যাপ খুললে — পুরো ফোনের স্ক্রিন না, শুধু map দেখানোর অংশটাই Viewport।

সবকিছু একসাথে:



Window → Viewport Transformation মানে Window-এর প্রতিটি বিন্দুকে Viewport-এ সঠিক অবস্থানে রূপান্তর করা।

2D Viewing Transformation Pipeline (ধাপক্রম)

এটা হলো পুরো প্রক্রিয়ার ধাপগুলো:

STEP 1: World Coordinate-এ Object তৈরি



STEP 2: Window নির্ধারণ (কোন অংশ দেখাবো?)



STEP 3: Clipping (Window-এর বাইরের অংশ বাদ দাও)



STEP 4: Window → Viewport Transformation (গণনা করো)



STEP 5: Device Coordinate-এ Viewport-এ দেখাও

Clipping (ক্লিপিং): Window-এর বাইরে যা আছে সেটা কেটে বাদ দেওয়া — যাতে শুধু নির্বাচিত অংশই দেখা যায়।

Window to Viewport Transformation

It may be possible that the size of the Viewport is much smaller or greater than the Window. In these cases, we have to increase or decrease the size of the Window according to the Viewport and for this, we need some mathematical calculations.

(x_w, y_w): A point on the window, and (x_v, y_v): Corresponding point on the viewport

We have to calculate the point (x_v, y_v)

Normalized Point on Window $\left(\frac{X_w - X_{wmin}}{X_{wmax} - X_{wmin}}, \frac{Y_w - Y_{wmin}}{Y_{wmax} - Y_{wmin}} \right)$

Normalized Point on Viewport $\left(\frac{X_v - X_{vmin}}{X_{vmax} - X_{vmin}}, \frac{Y_v - Y_{vmin}}{Y_{vmax} - Y_{vmin}} \right)$

3 Transformation-এর গণিত — সহজভাবে

সমস্যা:

Window-এর একটা বিন্দু (X_w, Y_w) — এটাকে Viewport-এ কোথায় রাখবো (X_v, Y_v)?

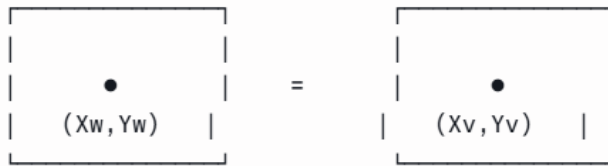
মূল নিয়ম:

Window-এ বিন্দুর আপেক্ষিক অবস্থান = Viewport-এ বিন্দুর আপেক্ষিক অবস্থান

মানে, বিন্দু যদি Window-এর মাঝখানে থাকে, সে Viewport-এরও মাঝখানে যাবে।

Window-এ অবস্থান:

Viewport-এ অবস্থান:



(আনুপাতিক অবস্থান একই)

Window to Viewport Transformation



From ① and ②, after calculating for x and y coordinate, we get

$$X_v = X_{vmin} + (X_w - X_{wmin}) S_x$$

$$Y_v = Y_{vmin} + (Y_w - Y_{wmin}) S_y$$

Where s_x is the scaling factor of x coordinate and s_y is the scaling factor of y coordinate

$$S_x = \frac{X_{vmax} - X_{vmin}}{X_{wmax} - X_{wmin}}$$

$$S_y = \frac{Y_{vmax} - Y_{vmin}}{Y_{wmax} - Y_{wmin}}$$

Scaling Factor (স্কেলিং ফ্যাক্টর):

Window ও Viewport-এর আকার আলাদা হতে পারে। তাই একটা **scaling factor** দরকার:

$$S_x = \frac{\text{Viewport-এর প্রস্থ}}{\text{Window-এর প্রস্থ}}$$

$$S_x = (X_{vmax} - X_{vmin}) / (X_{wmax} - X_{wmin})$$

$$S_y = \frac{\text{Viewport-এর উচ্চতা}}{\text{Window-এর উচ্চতা}}$$

$$S_y = (Y_{vmax} - Y_{vmin}) / (Y_{wmax} - Y_{wmin})$$

চূড়ান্ত সূত্র:

$$X_v = X_{vmin} + (X_w - X_{wmin}) \times S_x$$

$$Y_v = Y_{vmin} + (Y_w - Y_{wmin}) \times S_y$$

সূত্র বুঝি:

- **(X_w - X_{wmin})** → Window-এর শুরু থেকে বিন্দুটা কতদূর
 - **× S_x** → সেই দূরত্বকে Viewport-এর scale-এ রূপান্তর
 - **+ X_{vmin}** → Viewport-এর শুরুর জায়গা থেকে যোগ করো
-

4 উদাহরণ সমাধান (Step by Step)

দেওয়া আছে:

Window:	Viewport:
X _{wmin} = 20	X _{vmin} = 30
X _{wmax} = 80	X _{vmax} = 60
Y _{wmin} = 40	Y _{vmin} = 40
Y _{wmax} = 80	Y _{vmax} = 60

Window-এর বিন্দু: (X_w, Y_w) = (30, 80)

Viewport-এর বিন্দু বের করো: (X_v, Y_v) = ?

ধাপ ১: **S_x** বের করো

$$\begin{aligned} S_x &= (X_{vmax} - X_{vmin}) / (X_{wmax} - X_{wmin}) \\ &= (60 - 30) / (80 - 20) \\ &= 30 / 60 \\ &= 0.5 \end{aligned}$$

ধাপ ২: **S_y** বের করো

$$\begin{aligned} S_y &= (Y_{vmax} - Y_{vmin}) / (Y_{wmax} - Y_{wmin}) \\ &= (60 - 40) / (80 - 40) \\ &= 20 / 40 \\ &= 0.5 \end{aligned}$$

ধাপ ৩: **X_v** বের করো

$$\begin{aligned}
 X_v &= X_{vmin} + (X_w - X_{wmin}) \times S_x \\
 &= 30 + (30 - 20) \times 0.5 \\
 &= 30 + 10 \times 0.5 \\
 &= 30 + 5 \\
 &= 35
 \end{aligned}$$

ধাপ ৪: **Yv** বের করো

$$\begin{aligned}
 Y_v &= Y_{vmin} + (Y_w - Y_{wmin}) \times S_y \\
 &= 40 + (80 - 40) \times 0.5 \\
 &= 40 + 40 \times 0.5 \\
 &= 40 + 20 \\
 &= 60
 \end{aligned}$$

✓ উত্তর:

Window-এর বিন্দু $(X_w, Y_w) = (30, 80)$



Viewport-এর বিন্দু $(X_v, Y_v) = (35, 60)$

Example



Example: Let us assume:

- for window,
 $X_{wmin} = 20, X_{wmax} = 80,$
 $Y_{wmin} = 40, Y_{wmax} = 80.$
- for viewport,
 $X_{vmin} = 30, X_{vmax} = 60,$
 $Y_{vmin} = 40, Y_{vmax} = 60.$

Now a point (X_w, Y_w) be $(30, 80)$ on the window. We have to calculate that point on the viewport i.e (X_v, Y_v) .

Solution:

First of all, calculate the scaling factor of x coordinate S_x and the scaling factor of y coordinate S_y using the above-mentioned formula.

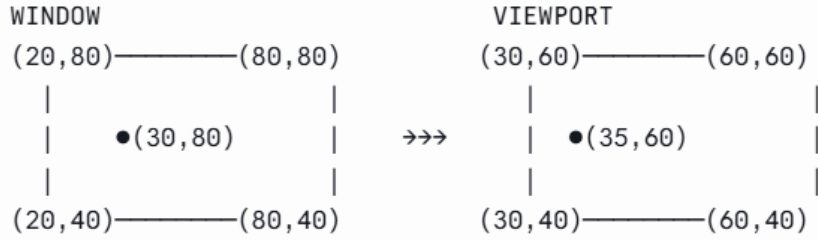
$$S_x = (60 - 30) / (80 - 20) = 30 / 60, \quad S_y = (60 - 40) / (80 - 40) = 20 / 40$$

Now calculate the point on the viewport (X_v, Y_v) .

$$X_v = 30 + (30 - 20) * (30 / 60) = 35, \quad Y_v = 40 + (80 - 40) * (20 / 40) = 60$$

So, the point on window $(X_w, Y_w) = (30, 80)$ will be $(X_v, Y_v) = (35, 60)$ on viewport

চিত্রে দেখি:



বিন্দু Window-এর বাম দিকে একটু ভেতরে।
Viewport-এও বাম দিকে একটু ভেতরে – অনুপাত একই।

5 Viewport-এর আকার ছোট বা বড় হলে কী হয়?

পরিস্থিতি

ফলাফল

Viewport = Window-এর সমান

হুবহু একই দেখাবে

Viewport < Window

ছবি ছোট হয়ে যাবে (zoom out)

Viewport > Window

ছবি বড় হয়ে যাবে (zoom in)

📱 উদাহরণ: Google Maps-এ zoom in করলে — একই Window-এর ছোট অংশ বড় Viewport-এ দেখাচ্ছে। Zoom out করলে বড় অংশ ছোটতে।

🔑 সব গুরুত্বপূর্ণ Terms একসাথে

Term	বাংলা	সহজ মানে
World Coordinate	বিশ্ব কো-অর্ডিনেট	object যেখানে তৈরি হয়
Device Coordinate	ডিভাইস কো-অর্ডিনেট	স্ক্রিনের pixel coordinate
Window	উইন্ডো	World-এ দেখার নির্বাচিত অংশ
Viewport	ভিউপোর্ট	স্ক্রিনে দেখানোর নির্দিষ্ট এলাকা
Clipping	ক্লিপিং	Window-এর বাইরের অংশ বাদ দেওয়া
Scaling Factor (Sx, Sy)	স্কেলিং ফ্যাক্টর	Window ও Viewport-এর অনুপাত

Transformation	রূপান্তর	এক coordinate থেকে অন্যতে নিয়ে যাওয়া
Xwmin/Xwmax	—	Window-এর X-এর সীমা
Xvmin/Xvmax	—	Viewport-এর X-এর সীমা

পুরো Lecture সারসংক্ষেপ

World Coordinate (কাল্পনিক জগৎ)



Window নির্ধারণ (কোন অংশ দেখাবো?)



Clipping (বাইরের অংশ বাদ)



Scaling Factor বের করো:

$$S_x = (X_{vmax} - X_{vmin}) / (X_{wmax} - X_{wmin})$$

$$S_y = (Y_{vmax} - Y_{vmin}) / (Y_{wmax} - Y_{wmin})$$



সূত্রে বসাও:

$$X_v = X_{vmin} + (X_w - X_{wmin}) \times S_x$$

$$Y_v = Y_{vmin} + (Y_w - Y_{wmin}) \times S_y$$



Device Coordinate-এ Viewport-এ দেখাও 



মনে রাখার সহজ উপায়

Window = তুমি কী দেখতে চাও (World-এ) **Viewport** = তুমি কোথায় দেখতে চাও (Screen-এ)

Transformation = Window → Viewport-এ বিন্দু সরানো **Scaling Factor** = দুটোর আকারের অনুপাত



চূড়ান্ত উদাহরণ: YouTube-এ ভিডিও দেখছো। ভিডিওর পুরো frame হলো World Coordinate। তুমি যে অংশটুকু দেখছো সেটা Window। তোমার স্ক্রিনে যে বাক্সে ভিডিও চলছে সেটা Viewport। Window to Viewport Transformation-ই ভিডিওকে সেই বাক্সে সঠিকভাবে ফিট করাচ্ছে!
